IPC144 – Introduction to C Programming
Practice Exercises
Week 6 - Arrays

Q1 – Write a C program to calculate average of an integer array.

Q2 – (Sales Commissions) Use a single-subscripted array to solve the following problem. A company pays its salespeople on a commission basis. The salespeople receive $200 per week plus 9% of their gross sales for that week. For example, a salesperson who grosses $3000 in sales in a week receives $200 plus 9% of $3000, or a total of $470. Write a C program (using an array of counters) that determines how many of the salespeople earned salaries in each of the following ranges (assume that each salesperson's salary is truncated to an integer amount):

a) $200–299
b) $300–399
c) $400–499
d) $500–599
e) $600–699
f) $700–799
g) $800–899
h) $900–999
i) $1000 and over

Q3 – (Bubble Sort) The bubble sort presented in Fig. 6.15 is inefficient for large arrays. Make the following simple modifications to improve the performance of the bubble sort.

a) After the first pass, the largest number is guaranteed to be in the highest-numbered element of the array; after the second pass, the two highest numbers are "in place," and so on. Instead of making nine comparisons on every pass, modify the bubble sort to make eight comparisons on the second pass, seven on the third pass and so on.

b) The data in the array may already be in the proper order or near-proper order, so why make nine passes if fewer will suffice? Modify the sort to check at the end of each pass whether any swaps have been made. If none has been made, then the data must already be in the proper order, so the program should terminate. If swaps have been made, then at least one more pass is needed.

Q4 – Write single statements that perform each of the following single-subscripted array operations:

a) Initialize the 10 elements of integer array counts to zeros.
b) Add 1 to each of the 15 elements of integer array bonus.
c) Read the 12 values of floating-point array monthlyTemperatures from the keyboard.
d) Print the five values of integer array bestScores in column format.

Q5 – Find the error(s) in each of the following statements:

a) Assume: char str[5];
```
scanf( "%s", str ); /* User types hello */
```

b) Assume: int a[3];
```
printf( "$d %d %d\n", a[ 1 ], a[ 2 ], a[ 3 ] );
```

c) double f[3] = { 1.1, 10.01, 100.001, 1000.0001 };

d) Assume: double d[2][10];
```
d[ 1, 9 ] = 2.345;
```

Q6 – (Duplicate Elimination) Use a single-subscripted array to solve the following problem. Read in 20 numbers, each of which is between 10 and 100, inclusive. As each number is read, print it only if it's not a duplicate of a number already read. Provide for the "worst case" in which all 20 numbers are different. Use the smallest possible array to solve this problem.

Q7 – What does the following program do?

```
/* What does this program do? */
#include <stdio.h>
#define SIZE 10

int whatIsThis( const int b[], int p ); /* function prototype */

/* function main begins program execution */
int main( void )
{
    int x; /* holds return value of function whatIsThis */

    /* initialize array a */
    int a[ SIZE ] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    x = whatIsThis( a, SIZE );

    printf( "Result is %d\n", x );
    return 0; /* indicates successful termination */
} /* end main */

/* what does this function do? */
int whatIsThis( const int b[], int p )
{
```

```
    /* base case */
    if ( p == 1 )
    {
        return b[ 0 ];
    } /* end if */
    else   /* recursion step */
    {

        return b[ p - 1 ] + whatIsThis( b, p - 1 );
    } /* end else */
} /* end function whatIsThis */
```

Q8 – (Dice Rolling) Write a program that simulates the rolling of two dice. The program should use rand to roll the first die and should use rand again to roll the second die. The sum of the two values should then be calculated. [Note: Since each die can show an integer value from 1 to 6, then the sum of the two values will vary from 2 to 12, with 7 being the most frequent sum and 2 and 12 the least frequent sums.] The following image shows the 36 possible combinations of the two dice. Your program should roll the two dice 36,000 times. Use a single-subscripted array to tally the numbers of times each possible sum appears. Print the results in a tabular format. Also, determine if the totals are reasonable; i.e., there are six ways to roll a 7, so approximately one-sixth of all the rolls should be 7.

	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12